第 8 课：向量化

吴恩达《Machine Learning Specialization》· Course 1 · Week 2 · Lesson 1

承上：第 7 课的模型 `f = w₁x₁ + w₂x₂ + … + b`，手写出来又长又啰嗦。 **这一课**：学一个能让代码又快又短的大杀器——**向量化（Vectorization）**。

一句话：把"一行行循环"变成"整个向量一次算完"。

为什么重要：不只写得短，还能**快好几个数量级**——现实里的模型可能有几千个特征、几百万个样本，向量化是必须的。

怎么用：从头往下读，代码格按 `Shift + Enter` 运行。

第 0 步：环境自检

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager
_candidates = ["Microsoft YaHei", "SimHei", "PingFang SC",
               "Noto Sans CJK JP", "Noto Sans CJK SC", "Droid Sans Fallback"]
_available = {f.name for f in font_manager.fontManager.ttflist}
_chosen = [c for c in _candidates if c in _available] or ["DejaVu Sans"]
plt.rcParams["font.sans-serif"] = _chosen
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["savefig.bbox"] = "tight"
print("本次使用中文字体：", _chosen)
print(" 环境就绪，可以继续啦！")
```

```
本次使用中文字体： ['Noto Sans CJK JP', 'Droid Sans Fallback']
 环境就绪，可以继续啦！
```

一、不向量化的写法：for 循环

对多元模型 `f = w·x + b`，新手的本能写法是循环：

In [2]:

```
# 手写循环版
w = np.array([0.1, 4.0, 20.0, -1.5])
x = np.array([1000, 3, 2, 10])
b = 50.0

f = 0.0
for i in range(len(w)):
    f = f + w[i] * x[i]
f = f + b
print("for 循环版 f =", f)
```

```
for 循环版 f = 187.0
```

能跑，但有两个问题：

1. **慢**——Python 循环对每个数都要处理一遍，大数据时非常慢；
2. **啰嗦**——每加一个特征都得再写一遍循环。

二、向量化：一行搞定

同样的事，用 **numpy 的点积** `np.dot(w, x)` 一行做完：

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

- `\mathbf{w} \cdot \mathbf{x}` 读作"w 点乘 x"，就是**逐项相乘再求和**；
- 结果和循环版**一模一样**，但写法短得多。

In [3]:

```
# 向量化版
w = np.array([0.1, 4.0, 20.0, -1.5])
x = np.array([1000, 3, 2, 10])
b = 50.0

f = np.dot(w, x) + b      # ← 一行，整串算完
print("向量化版 f =", f)

# 验证两种写法结果相同
print("和循环版一致吗？", np.isclose(f, 0.1*1000+4.0*3+20.0*2+(-1.5)*10+b))
```

```
向量化版 f = 187.0
和循环版一致吗？ True
```

三、到底快多少？跑一跑看

为了看清楚差别，我们用**几十万个数**来做同样的乘加，比较循环版和向量版的耗时。

In [4]:

```python
import time

N = 500_000                 # 50 万个元素
w = np.random.rand(N)
x = np.random.rand(N)

# 循环版
start = time.time()
f_loop = 0.0
for i in range(N):
    f_loop += w[i] * x[i]
t_loop = time.time() - start

# 向量版
start = time.time()
f_vec = np.dot(w, x)
t_vec = time.time() - start

print(f"循环版 耗时: {t_loop:.4f} 秒")
print(f"向量版 耗时: {t_vec:.4f} 秒")
print(f"加速约 {t_loop / t_vec:.0f} 倍")
```

```
循环版 耗时: 0.0773 秒
向量版 耗时: 0.0004 秒
加速约 185 倍
```

看：向量版往往快上**几十到上百倍**——数字越大差别越夸张。

为什么差这么多？ 因为 `np.dot` 是在 **C 语言层面**（以及 CPU 的 SIMD 并行）一次算完的，而 Python 的 `for` 每一步都要来回解释、慢吞吞。

向量化的两个好处：

1. **代码更短、更好读**；
2. **快得惊人**——这在真实项目（几千维特征、几百万样本）里，是"几分钟 vs 几小时"的差别。

四、把向量化用到模型和梯度下降上

模型：

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

梯度下降的更新式（下一课细讲），也能全部向量化——**一次算出所有 w 的梯度**：

$$\mathbf{w} := \mathbf{w} - \alpha \cdot \mathbf{d_w} \quad \text{其中 } \mathbf{d_w} = \frac{1}{m}\sum_{i=1}^{m}\left(f^{(i)} - y^{(i)}\right)\mathbf{x}^{(i)}$$

- `\mathbf{d_w}` 是一个**向量**，长度等于特征数；
- `w` 和 `d_w` **逐项相减**（向量的减法），一次更新全部权重。

这就是向量化的完整威力：一条公式，更新所有参数。

五、小结

1. **向量化** = 把循环换成向量运算（numpy 的 `np.dot`、逐项加减）。
2. **模型**：`f = np.dot(w, x) + b`，一行搞定。
3. **两个好处**：代码短、速度快（几十到上百倍）。
4. **应用**：梯度下降也能向量化，一条公式更新所有 w。

六、反转课堂 · 课后作业

1. 用 numpy 写一行向量化代码，计算 `w = [2, 3, 4]`、`x = [1, 1, 1]` 的 `f = w·x + b`，其中 b = 1。结果是多少？
2. 为什么向量版比 for 循环快这么多？（提示：在哪一层语言里跑？）
3. 如果一个模型有 **5 个特征**，`np.dot(w, x)` 里的 w 和 x 分别是长度多少的向量？

下一课（第 9 课）：多元梯度下降——把这套向量化真正用到训练里。